Flight & Hotel Booking Platform — Technical Build Brief (Duffel-based)

Project Goal

Build a booking web app where users search flights by origin/destination/date, see live bookable
offers, and check out via either online card payment or a cash-hold-and-pay-later flow. Phase 2 extends the same architecture to hotel bookings using Duffel's Stays API.

Use this document as a build prompt for yourself or an AI coding assistant (Cursor, Copilot, Claude, etc.) — each section names the exact technology, why it was chosen, and what it's responsible for.

1. Backend Framework — Django + Django REST Framework

Why: Matches your existing Python skillset; Amadeus itself ships a reference booking app in Django, and DRF gives you serializers/viewsets that map cleanly onto Duffel's REST resources (offer requests, offers, orders, payments).

Responsibilities:

- `flights/` app: models and views for `OfferRequest`, `Offer` (cached from Duffel, not source of truth), `Order`, `Payment`.
- `hotels/` app (Phase 2): mirrors `flights/` against Duffel Stays.
- `payments/` app: Stripe integration for online, hold/cash-confirmation logic for cash.
- `accounts/` app: customer accounts + an internal "agent/staff" role that can confirm cash payments and manage hold-order expiry.

2. Flight/Hotel Data Provider — Duffel API

Why: REST-first, instant sandbox access (no accreditation wait like Amadeus Enterprise), covers both flights (300+ airlines) and hotels (Stays, 2M+ properties) under one account and one auth model, and has a hold-order primitive that matches your cash-payment requirement natively.

Key resources to integrate, in order:

1. `POST /air/offer_requests` — create a search (origin, destination, dates, passengers).
2. `GET /air/offers` / `GET /air/offers/{id}` — list/refresh live offers for that request.
3. `POST /air/orders` — book. Two modes:
    - Instant: `type: instant`, include `payment` object → charges immediately.

- Hold: `type: hold`, no `payment` object → reserves space, returns an expiry timestamp.

4. `POST /air/payments` — pay for a previously held order before it expires (used for your cash flow once staff confirms receipt).

Auth: Bearer access token in `Authorization: Bearer $TOKEN`, plus a required `Duffel-Version` header on every request. Use `duffel_test_...` tokens in development (created from the Duffel dashboard in "test mode") and swap to a live token only in production.

Client module: write a single `services/duffel_client.py` wrapping `requests` (no official Django SDK needed — Duffel is plain REST) so all four calls above live in one place and are easy to unit test with mocked responses.

3. Database — PostgreSQL

Why: Relational integrity for orders/payments/passengers; JSONB columns let you store the raw
Duffel offer/order payloads for auditability without over-normalizing volatile provider data.

Key tables: `orders` (status: pending_payment / hold_pending_cash / confirmed / expired / cancelled), `payments`, `passengers`, `offer_snapshots` (JSONB cache of the priced offer used at booking time, so you always know exactly what was shown to the customer).

4. Background Jobs — Celery + Redis

Why: Hold orders expire; you need a scheduled task to check for near-expiry cash bookings and alert staff/customers, and to auto-cancel expired holds. Also useful later for sending booking confirmation emails without blocking the request cycle.

Tasks: `expire_stale_hold_orders`, `send_booking_confirmation`, `notify_expiring_cash_hold`.

5. Online Payments — Stripe

Why: Mature EU/Germany support, strong Django integration (`dj-stripe` or direct `stripe-python`), handles 3D Secure/SCA which is mandatory for EU card payments.

Flow: customer selects "pay online" → create Stripe PaymentIntent for the offer's total → on success, immediately call Duffel's instant order-with-payment endpoint → mark `Order.status = confirmed`.

6. Cash Payment Flow — Duffel Hold Orders + internal admin action

Why: No third-party cash processor needed — this is an internal state machine on top of Duffel's native hold-order feature.

Flow: customer selects "pay cash" → create Duffel hold order → `Order.status = hold_pending_cash`, store expiry → staff member confirms cash received via an internal Django admin action or a small staff-only view → trigger `POST /air/payments` against the held order → mark `confirmed`.

7. Frontend — Django templates + HTMX (or React if you want a SPA)

Why: Given the priority is backend correctness (live pricing, payment branching) rather than a highly interactive UI, Django templates + HTMX let you ship fast without maintaining a separate
frontend build pipeline. If you want a more app-like feel for your portfolio, swap this for a React/Next.js frontend consuming your DRF API — either works with the same backend.

8. Auth — Django's built-in auth + `django-allauth`

Why: Handles customer signup/login and a separate staff/agent permission group for the cash-confirmation admin actions, without building auth from scratch.

9. Containerization & Deployment — Docker + Kubernetes (AKS)

Why: Matches your existing Azure/AKS experience and makes this a stronger portfolio piece for DevOps/cloud roles, not just a working app.

Setup: separate containers for Django app, Celery worker, Celery beat (scheduler), Redis, Postgres (managed Azure Database for Postgres in production). Use a Helm chart or plain manifests
you already know from your Kubernetes labs.

10. CI/CD — GitHub Actions

Why: Free, integrates directly with GitHub where the code lives; run `pytest` + linting on every PR, build/push Docker images, and deploy to AKS on merge to main.

11. Testing — pytest + Duffel test mode

Why: Duffel's test mode uses a sandbox airline ("Duffel Airways") plus documented test credit card numbers for Stripe-style payment testing, so your CI can run real integration tests against a stable sandbox instead of mocking everything.

Test routes: use Duffel's documented test routes (e.g., LHR→JFK on Duffel Airways) for reliable, repeatable sandbox results, since real airline sandboxes can be flaky.

12. Monitoring — Sentry (errors) + Django logging to stdout for Kubernetes log aggregation

Why: Cheap, fast to wire in, and gives you visibility into failed Duffel calls or expired holds without building a custom dashboard.

Build Order Checklist

1. Django project scaffold + Postgres + Duffel test token in `.env`.
2. `duffel_client.py` wrapping offer_requests/offers/orders/payments.
3. Search view → offer_requests + offers, cache response.

4. Booking view: branch online (Stripe + instant order) vs cash (hold order).
5. Celery worker: expire stale holds, send confirmations.
6. Staff view: confirm cash payment → trigger Duffel payment call.
7. Dockerize, deploy to AKS, wire GitHub Actions CI.
8. Repeat steps 2-4 against Duffel Stays for hotels (Phase 2).